

1 Lagrangian2D 求解器

1.1 实验概述

SPH 是一种拉格朗日视角的无网格方法，通过离散的粒子集合来近似流体方程。接下来我将详细描述求解器的核心算法实现，包括核函数的选择、物理量的计算流程以及数值积分与边界处理方法。

1.2 核心算法实现

1.2.1 核函数

在 SPH 方法中，核函数 $W(r, h)$ 用于将离散粒子的属性平滑插值到连续空间。为了保证数值稳定性和物理准确性，我们针对不同的物理量计算选择了不同的核函数：

- **Poly6 核函数**: $W_{poly6}(r, h) = \frac{4}{\pi h^8} (h^2 - r^2)^3$, $0 \leq r \leq h$ 该核函数主要用于密度计算，在 $r = 0$ 处平滑，适合标量插值，但在 $r \rightarrow 0$ 时梯度趋于零，不适合计算压力力，会导致粒子聚集。
- **Spiky 核函数梯度**: $\nabla W_{spiky}(r, h) = -\frac{30}{\pi h^5} (h - r)^2 \frac{\mathbf{r}}{r}$, $0 \leq r \leq h$ 该核函数用于压力梯度力计算，在 $r \rightarrow 0$ 时梯度不为零，能提供强大的排斥力，有效防止粒子聚集。
- **Viscosity 核函数拉普拉斯**: $\nabla^2 W_{viscosity}(r, h) = \frac{40}{\pi h^5} (h - r)$, $0 \leq r \leq h$ 这个核函数主要用于压力梯度力计算。

1.2.2 求解器流程 (Solver Loop)

求解器的 `solve()` 函数实现了单步物理仿真，严格遵循以下六个主要步骤：

1. 计算密度: 使用 SPH 核函数计算每个粒子的密度。系统先通过 `mPs.updateBlockInfo()` 将粒子分配到网格中执行邻域搜索。然后计算密度: $\rho_i = \sum_j m_j W_{poly6}(\mathbf{r}_i - \mathbf{r}_j, h)$ 为了防止避免后续除以零，并防止密度过低导致的数值问题，我们对计算出的密度进行了下限截断: $\rho_i = \max(\rho_i, \rho_{rest})$ 。
2. 根据密度计算压力。使用 Tait 状态方程: $p_i = k \left(\left(\frac{\rho_i}{\rho_0} \right)^\gamma - 1 \right)$ ，同时为了避免拉伸不稳定性，我们将压力限制为非负值: $p_i = \max(0, p_i)$ ，防止粒子在低密度区域产生吸力。
3. 计算压力梯度和粘性力，进而得到加速度:
 - 1) 压力梯度力将推动粒子从高压区流向低压区: $\mathbf{F}_i^{pressure} = - \sum_j m_j m_i \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla W_{spiky}(\mathbf{r}_{ij}, h)$;
 - 2) 粘性力来模拟流体内部摩擦: $\mathbf{F}_i^{viscosity} = \mu \sum_j m_j \frac{\mathbf{v}_j - \mathbf{v}_i}{\rho_j} \nabla^2 W_{viscosity}(\mathbf{r}_{ij}, h)$;
 最后根据牛顿第二定律计算加速度: $\mathbf{a}_i = \frac{\mathbf{F}_i^{pressure} + \mathbf{F}_i^{viscosity}}{m_i} + \mathbf{g}$ 。
4. 使用欧拉积分更新粒子的速度和位置:

$$\begin{aligned}\mathbf{v}_i(t + \Delta t) &= \mathbf{v}_i(t) + \mathbf{a}_i(t) \Delta t \\ \mathbf{x}_i(t + \Delta t) &= \mathbf{x}_i(t) + \mathbf{v}_i^{XSPH}(t + \Delta t) \Delta t\end{aligned}$$

为了提高流体的连贯性并防止粒子穿透，我们在位置更新时引入了 XSPH 人工粘性修正速度 $\mathbf{v}_i^{XSPH}$: $\mathbf{v}_i^{XSPH} = \mathbf{v}_i + \epsilon \sum_j \frac{2m_j}{\rho_i + \rho_j} (\mathbf{v}_j - \mathbf{v}_i) W_{poly6}(\mathbf{r}_{ij}, h)$

5. 我们接下来进行边界检查，当粒子超出容器边界时，将其位置强制拉回边界内，并反转其垂直于边界的速度分量，同时乘以衰减系数（模拟非完全弹性碰撞）

6. 最后更新粒子的空间索引。在位置更新后，粒子的坐标发生了变化，因此需要重新计算其所在的网格块 ID (blockId)，以便在下一帧的邻域搜索中能正确找到该粒子。

1.3 喷泉效果

接下来我们查看喷泉的效果

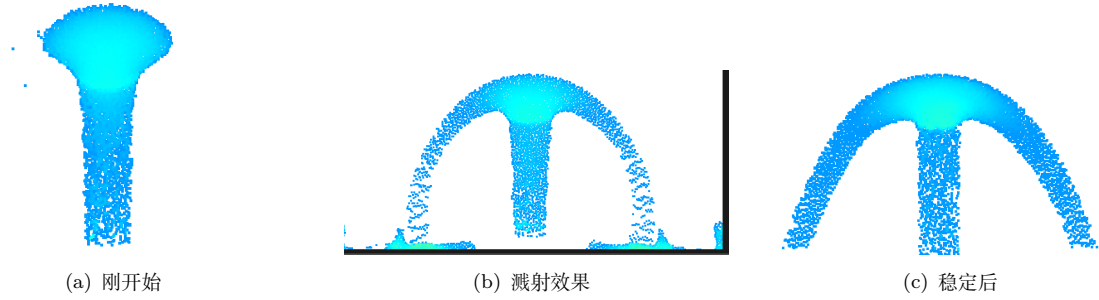


Figure 1: 不同阶段的喷泉效果

1.4 总结

本实验成功实现了一个功能完整的二维 SPH 流体求解器。通过引入 Spiky 核函数计算压力梯度、限制负压以及应用 XSPH 人工粘性，有效解决了粒子聚集和穿透等常见问题，模拟出了稳定且真实的流体效果。

2 Eulerian2D 求解器

2.1 实验原理

本实验的理论基础是不可压缩 Navier-Stokes 方程，其动量方程描述了流体速度随时间的变化规律：

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \quad (1)$$

同时，流体必须满足不可压缩条件，即速度场的散度为零：

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

在求解策略上，我们采用了稳定流体方法。即利用算子分裂法，将一个时间步长内的求解过程拆分为三个主要步骤：首先是平流，处理流体自身的输运项 $(\mathbf{u} \cdot \nabla) \mathbf{u}$ ；其次是外力，计算如重力、浮力等外部作用力 $\mathbf{f}$ ；最后是投影，求解压力项 $-\frac{1}{\rho} \nabla p$ 并修正速度场，强制其满足不可压缩约束。

2.2 实现步骤与核心代码

2.2.1 平流项

平流项的计算采用了半拉格朗日法，基本逻辑是“回溯”。对于网格上的每一个点 $\mathbf{x}$ ，我们根据当前的速度场逆向追踪，找到它在上一时刻的位置 $\mathbf{x}_{old} = \mathbf{x} - \mathbf{u}(\mathbf{x})\Delta t$ 。然后，通过插值获取该位置在上

一时刻的物理量 $q_{old}(\mathbf{x}_{old})$ ，并将其赋值给当前点作为新的物理量 $q_{new}(\mathbf{x})$ 。

```

1 // 水平速度分量的核心逻辑，其余的垂直速度分量、密度场、温度场也是类似的逻辑
2 // 找到当前U分量的世界坐标位置
3 glm::vec2 pos(i * dx, (j + 0.5) * dx);
4 glm::vec2 oldPos = pos;
5 // 回溯寻找上一时刻的位置
6 pos = mGrid.semiLagrangian(pos, dt);
7 // 处理固体边界：如果回溯点在固体内部，则使用原位置
8 if (mGrid.inSolid(pos))
9     pos = oldPos;
10 // 插值获取上一时刻该位置的速度值
11 targetU(i, j) = mGrid.getVelocityX(pos);

```

2.2.2 外力项

在本实验中，我们主要模拟烟雾受热上升的现象，因此引入了基于 Boussinesq 近似的热浮力。具体来说，流体密度越高，受到的重力越大（方向向下）；而温度越高，受到的浮力越大（方向向上）。浮力公式可以表示为 $f_{buoy} = -\alpha\rho + \beta(T - T_{ambient})$ 。计算出的浮力将直接作用于垂直速度分量 V 上，从而驱动烟雾产生自然的上升运动。

```

1 // 核心逻辑：
2 // 如果当前面是固体边界（例如墙壁），则跳过，不施加力
3 if (mGrid.isSolidFace(i, j, MACGrid2d::Y))
4     continue;
5
6 // 计算当前垂直速度分量 V 所在的世界坐标位置
7 glm::vec2 pos((i + 0.5) * dx, j * dx);
8
9 // 计算浮力并更新速度
10 // 根据  $\mathbf{v} = \mathbf{v}_0 + \mathbf{a} * \mathbf{t}$ ，将浮力产生的速度增量加到当前速度上
11 mGrid.mV(i, j) += dt * mGrid.getBoussinesqForce(pos);

```

2.2.3 投影项

投影步骤是求解器中最关键也最耗时的部分，其目的是消除速度场的散度，确保流体体积守恒。根据亥姆霍兹分解定理，任何向量场都可以分解为一个无散场和一个梯度场。我们需要求解压力泊松方程：

$$\nabla^2 p = \frac{\rho}{\Delta t} (\nabla \cdot \mathbf{u}^*) \quad (3)$$

其中 $\mathbf{u}^*$ 是经过平流和外力计算后得到的中间速度场。在具体实现中，我们将这一过程细分为三个子步骤：

首先是构建线性方程组。我们将泊松方程离散化为 $A\mathbf{x} = \mathbf{b}$ 的形式。其中，右端项 $\mathbf{b}$ 向量存储了当前速度场在每个网格单元处的负散度。如果某处的散度为正（流体流出），说明该处压力需要降低以吸入流体；反之则需要升高压力。矩阵 A 则是五点差分格式的离散拉普拉斯算子，描述了网格点与其邻居之间的压力耦合关系。

其次是求解线性方程组。由于矩阵 A 是一个大规模的稀疏对称正定矩阵，直接求逆在计算上是不可行的。因此，我们采用了预处理共轭梯度进行迭代求解。该算法通过不断修正搜索方向和残差，逐步逼近真实的压力解 p 。我们在代码中设置了最大迭代次数（200 次）和容差阈值（ $1e-5$ ）。

最后是修正速度场。一旦求得压力场 p ，我们就可以计算其梯度，并利用压力梯度来修正中间速度场 $\mathbf{u}^*$ ，从而得到最终的无散速度场 $\mathbf{u}_{new}$ ：

$$\mathbf{u}_{new} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p \quad (4)$$

2.3 实验结果

最终实现的求解器能够稳定运行，成功模拟出了流体的平流扩散效果。我们可以清晰地观察到热气体带动烟雾上升的过程，且上升速度与温差成正比，符合物理规律。经过投影步骤的处理，流体表现出了良好的体积守恒特性，没有出现明显的压缩或膨胀现象。特别是在修复了边界问题后，流体能够正确地绕过障碍物，并在障碍物后方形成逼真的涡流，展现了流体运动的复杂细节。

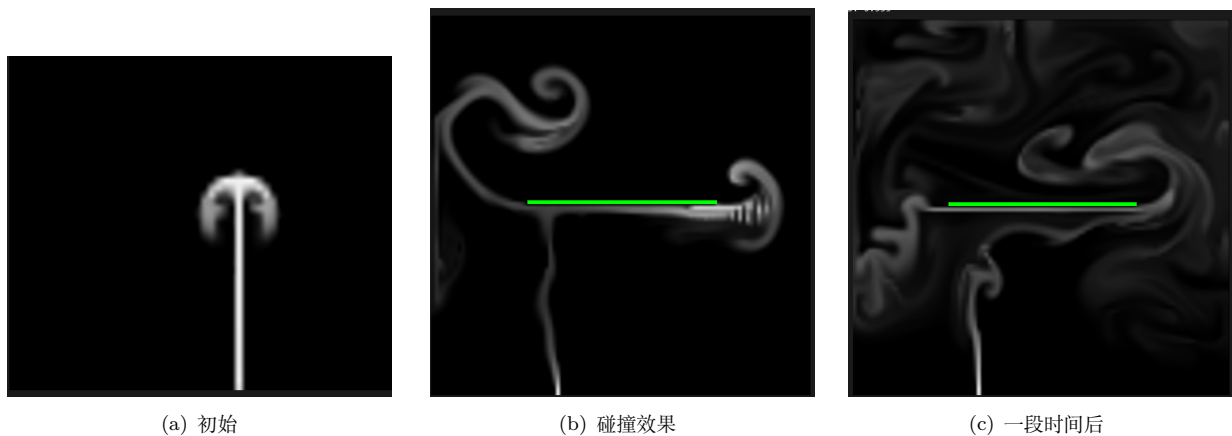


Figure 2: 不同阶段的烟雾效果

3 仓库连接

https://github.com/LoveGump/nku_CG2_Fluid_sim.git (已私密)

4 实验总结

本次实验成功实现了一个基于物理的二维欧拉流体求解器。通过亲手编写代码，我们不仅加深了对 Navier-Stokes 方程数值解法的理解，更在实践中体会到了理论模型与数值实现之间的微妙差异。特别是通过解决边界处的数值耗散问题，我们深刻认识到边界条件的处理对于模拟结果真实性的重要影响。虽然半拉格朗日法具有无条件稳定的优点，但其固有的数值耗散仍然是一个值得改进的地方，未来可以尝试引入 MacCormack 方法或 BFECC 方法，以进一步提高模拟的细节保留度。